Containerization

Enrique Calderon

LIDSoL

Semester 2026-2

Containerization Overview

Goal

Attendees create a `Containerfile`, build the image with Podman, and run it locally with exposed ports.

Duration

90 minutes

Introduction to Containerization

- Containers are lightweight, portable units that package an application and its dependencies together.
- They ensure consistency across different environments (development, staging, production).

Podman vs Docker

- Podman is a daemonless container engine.
- Rootless by default for better security.
- Command-line compatibility (`alias docker=podman`).
- Ecosystem integrates natively with `systemd` via Quadlet.

Building the Containerfile

A `Containerfile` is synonymous with a `Dockerfile`.

```
FROM python:3.14-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py app.py

# Default command using wait-server or gunicorn
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000",
↪  "app:app"]
```

Building the Container Image

Use the `podman build` command locally.

```
# Don't forget the dot at the end!
podman build -t my-flask-app:latest .
```

Verify your image:

```
podman images
```

Running the Container

Run your newly built image locally, mapping the internal port (5000) to your host port (5000).

```
podman run -d \
  --name flask_service \
  -p 5000:5000 \
  --env-file .env \
  my-flask-app:latest
```

Validating the Build

Check container status and logs:

```
podman ps
podman logs flask_service
```

Access the service locally:

```
curl http://127.0.0.1:5000/ -X POST \
  -H "Content-Type: application/json" \
  -d '{"secret_key": "supersecretkey"}'
```